REAL-TIME AIR QUALITY INDEX (AQI) AGGREGATOR & MAPPING SYSTEM

A PROJECT REPORT

Submitted by

DHIVAKARAN S (310624148019)

in partial fulfillment for the award of the Degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



EASWARI ENGINEERING COLLEGE,

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled “REAL-TIME AIR QUALITY INDEX (AQI) AGGREGATOR & MAPPING SYSTEM” is the Bonafide work of “DHIVAKARAN S (310624148019)”, who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT

Dr. Justin Dhas Y
Professor & Head
Department of CSE (AI & ML)
Easwari Engineering College

SUPERVISOR

Ms. Divyaanjali
Assistant Professor
Department of CSE (AI & ML)
Easwari Engineering College

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled “REAL-TIME AIR QUALITY INDEX (AQI) AGGREGATOR & MAPPING SYSTEM” submitted by Mr./Ms. Dhivakaran S (Reg. No. 310624148019) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF ENGINEERING was conducted on 22-10-2026 (Date) in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner	External Examiner
Name : _____	Name : _____
Designation : _____	Designation : _____
Address : Easwari Engineering College, Ramapuram, Chennai.	Address : _____ _____
Date: 22-10-2026	Date: 22-10-2026

ABSTRACT

Atmospheric pollution has emerged as one of the most critical threats to public health and global ecological equilibrium. Urban centers and industrial corridors experience severe concentrations of harmful airborne pollutants, including fine particulate matter (PM2.5), coarse particulate matter (PM10), nitrogen dioxide (NO2), sulfur dioxide (SO2), carbon monoxide (CO), and ground-level ozone (O3). Despite the proliferation of environmental monitoring stations worldwide, access to verified, unified, and actionable air quality telemetry remains substantially fragmented. Existing municipal and commercial portals frequently present delayed data, obscure raw concentrations without standardized sub-indexing, or lack dynamic geospatial mapping capabilities.

To overcome these critical barriers, this project presents the Real-Time Air Quality Index (AQI) Aggregator & Mapping System—an end-to-end, production-grade full-stack web platform built upon the MERN architectural paradigm (MongoDB Atlas, Express.js, React 18, and Node.js). The system interfaces directly with the international OpenAQ v3 API infrastructure via a scheduled, fault-tolerant background ingestion engine orchestrating node-cron tasks. A multi-tier normalization algorithm converts disparate raw physical pollutant concentrations (µg/m³ and ppm) into standardized Air Quality Index sub-indices adhering strictly to the Indian Central Pollution Control Board (CPCB) standard for domestic stations and the United States Environmental Protection Agency (US EPA) standard internationally.

The architecture incorporates bidirectional WebSocket synchronization powered by Socket.IO, enabling real-time telemetry pushes to client dashboards without page reloads. The presentation layer features high-performance interactive geospatial GIS mapping utilizing Leaflet and OpenStreetMap, rendering dynamic heatmaps, station boundary pins, and pollutant distribution overlays. Furthermore, historical telemetry is indexed in MongoDB Atlas to facilitate multi-interval timeseries trend visualisations (24-hour, 7-day, and 30-day analytics) rendered via SVG-based Recharts, alongside automated health advisories and threshold alert notifications.

Rigorous empirical testing confirms sub-second API latency, resilient rate-pacing compliance with external APIs, zero synthetic data fallback, and seamless cross-platform responsiveness. The resulting application equips citizens, researchers, and public health officials with reliable, transparent, and immediate environmental intelligence.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal Dr.P.Deiva Sundari and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department, Department of COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING), for the continuous encouragement and support.

We are extremely thankful to our Project Guide, Ms. Divyaanjali, Assistant Professor, Department of COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING) for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING) for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	iv
	ACKNOWLEDGEMENT	v
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	2
	1.3 OBJECTIVE	3
	1.4 SCOPE OF THE PROJECT	4
	1.5 MOTIVATION	4
2	**RELATED WORK**	**5**
	2.1 LITERATURE REVIEW	5
	2.2 EXISTING SYSTEM	6
	2.3 PROPOSED SYSTEM	7
3	**SYSTEM ANALYSIS**	**9**
	3.1 SYSTEM REQUIREMENTS	9
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	10
4	**SYSTEM DESIGN**	**11**
	4.1 SYSTEM ARCHITECTURE	11
	4.2 MODULE DESCRIPTION	12
	4.4 CLASS DIAGRAM	14
	4.5 SEQUENCE DIAGRAM	15
5	**IMPLEMENTATION**	**17**

	5.1 IMPLEMENTATION OVERVIEW	17
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	18
6	**CONCLUSION & FUTURE WORK**	**22**
	6.1 CONCLUSION	22
	6.2 FUTURE ENHANCEMENTS	23
APPENDIX	**SOURCE CODE**	**24**
	1. Project Structure	24
	2. Backend Package Installation	25
	3. Database Connection	26
	4. Environment Variables	27
	5. User Model	28
	6. Password Hashing and Registration	29
	7. Login and JWT Generation	30
	8. Authentication Middleware	31
	9. Create API	32
	10. Read API	33
	11. Update	35
	12. Delete	36
	13. Express Routes	37
	14. Main Server File	38
	15 React Login Component	40
	16. Admin Dashboard Logic	42
	17. REST API Request and	44

Response Examples

SCREENSHOTS OF THE PROJECT 47

REFERENCES 50

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Air pollution has escalated into one of the paramount environmental and public health emergencies of the twenty-first century. Rapid industrialization, explosive vehicular density, urbanization, and agricultural residue incineration have introduced severe loads of particulate matter and chemical toxicants into the troposphere. Inhalation of criteria atmospheric pollutants—specifically fine particulate matter (PM2.5), respirable particulate matter (PM10), nitrogen oxides (NOx), sulfur dioxide (SO2), carbon monoxide (CO), and tropospheric ozone (O3)—is directly correlated with acute respiratory infections, chronic obstructive pulmonary disease (COPD), ischemic heart disease, and premature mortality.

To quantify and communicate atmospheric pollutant concentrations in an easily understandable format for the public and policy administrators, environmental authorities utilize the Air Quality Index (AQI). The AQI translates complex concentrations of multiple pollutants into a singular, dimensionless numerical index divided into graded, color-coded health risk categories. For instance, the Indian Central Pollution Control Board (CPCB) defines an AQI scale ranging from 0 to 500 across six classifications: Good (0–50), Satisfactory (51–100), Moderate (101–200), Poor (201–300), Very Poor (301–400), and Severe (401–500). Similarly, international jurisdictions apply the United States Environmental Protection Agency (US EPA) standard.

The Real-Time Air Quality Index (AQI) Aggregator & Mapping System engineered in this project establishes a modern, full-stack, distributed environmental telemetry solution. By synchronizing with OpenAQ v3 API data feeds, applying rigorous mathematical breakpoint normalizations, executing compound spatial indexing in MongoDB Atlas, and delivering reactive visual analytics via React and Leaflet, the system delivers an authoritative, open, and responsive telemetry platform.

1.2 PROBLEM DEFINITION

Despite significant investments in physical continuous ambient air quality monitoring stations (CAAQMS) globally, modern air pollution data dissemination suffers from four structural deficits:

1. Data Fragmentation Across Disparate Silos: Environmental data is maintained across multiple governmental portals, regional databases, and proprietary vendor systems with incompatible access protocols and varying schema formats.

2. Ambiguity in Mathematical Indexing: Third-party feeds frequently report raw concentration metrics (e.g., $\mu g/m^3$ or ppm) without computing standardized sub-indices, leaving end users incapable of interpreting actual health severity.

3. Latency in Client Telemetry: Many legacy web dashboards rely on cumbersome manual browser reloads or unoptimized polling queries, causing severe server load and failing to deliver immediate notifications when pollution levels breach critical thresholds.

4. Deficits in Geospatial Context: Numerical tables fail to convey the geographical dispersion of air masses and pollution plumes, hindering spatial awareness and regional comparison.

1.3 OBJECTIVE

The principal objectives of the Real-Time AQI Aggregator & Mapping System encompass:

• Real-Time Ingestion Architecture: Engineer an automated, scheduled background pipeline querying OpenAQ v3 API endpoints using node-cron with adaptive request throttling, rate-limit safeguards, and fault-tolerant backoff algorithms.

• Standardized Multi-Pollutant AQI Engine: Formulate and implement mathematical normalizers calculating sub-indices for PM2.5, PM10, NO2, SO2, CO, and O3 following both Indian CPCB linear interpolation equations and US EPA regulatory standards.

• Geospatial GIS Visualization: Construct an interactive mapping interface with Leaflet and OpenStreetMap rendering custom color-graded station markers, interactive popups, and zoom-aware geospatial bounding.

• Duplex Live Updates: Integrate Socket.IO WebSocket pipelines enabling real-time broadcast of newly ingested environmental telemetry directly to connected client dashboards without browser refresh cycles.

• Timeseries Historical Analytics: Establish MongoDB timeseries schemas and indexing structures to deliver 24-hour, 7-day, and 30-day pollution trend graphs rendered via SVG Recharts.

• Proactive Health Advisories & Alerting: Formulate automated medical safety advisories corresponding to real-time pollutant levels and maintain user-customizable threshold breach alerts.

1.4 SCOPE OF THE PROJECT

The scope of this project spans the complete software development lifecycle of an environmental monitoring ecosystem, encompassing backend API microservices, distributed database schemas, asynchronous ingestion workers, and a reactive front-end single-page application. The platform provides global monitoring coverage with specialized regional prioritization for Indian CAAQMS stations. System boundaries strictly mandate the ingestion and display of 100% verified, real-world environmental telemetry, eliminating synthetic or simulated mock fallbacks. The system is designed for deployment across scalable containerized or serverless hosting environments.

1.5 MOTIVATION

Clean air is a fundamental human right. Nonetheless, over 90% of the global population breathes air exceeding World Health Organization (WHO) air quality guidelines. Accurate, transparent, and immediate air quality intelligence empowers individuals to take protective measures—such as wearing particulate respirators or activating HEPA purification—while equipping researchers and policymakers with empirical telemetry necessary for regulatory interventions. The intersection of modern full-stack web technologies, distributed cloud datastores, and open environmental APIs provides a compelling opportunity to build an impactful, open-access public health intelligence asset.

CHAPTER 2
RELATED WORK

2.1 LITERATURE REVIEW

Academic research and industry initiatives in ambient air quality monitoring have evolved significantly over the past decade:

1. Centralized Environmental Agency Systems: National agencies such as the Central Pollution Control Board (CPCB) in India and the US EPA manage extensive networks of CAAQMS stations. While these databases constitute authoritative ground truth, their public-facing web portals frequently suffer from rigid user interfaces, high server response times during peak pollution events, and lack of extensible RESTful APIs for third-party developers.

2. OpenAQ Global Initiative: OpenAQ operates as a non-profit organization providing a centralized open-source data platform aggregating ambient air quality telemetry from thousands of government and research stations worldwide. The introduction of the OpenAQ v3 API provides standardized REST endpoints, spatial bounding-box lookups, and sensor parameter definitions. However, OpenAQ deliberately provides raw physical concentrations rather than calculated AQI values, requiring downstream applications to compute index transformations.

3. Commercial Air Quality Platforms: Platforms like IQAir and BreezoMeter offer consumer-oriented visual dashboards and mobile apps. However, these systems rely heavily on proprietary mathematical modeling, proprietary sensor networks, and commercial subscription paywalls, restricting algorithmic transparency and raw research verification.

4. Web-Based Geospatial Visualization Frameworks: Research by environmental informatics scholars demonstrates that combining vector tile mapping (e.g., Leaflet, Mapbox) with reactive component architectures significantly enhances user risk perception compared to static tabular displays. Providing immediate spatial and temporal context facilitates intuitive public comprehension.

2.2 EXISTING SYSTEM

Existing air quality information platforms exhibit several key operational and technical drawbacks:

• Inconsistent Normalization: Different portals compute indices using incompatible formulas without disclosing the dominant pollutant or active breakpoint tables, resulting in conflicting readings for identical coordinates.

• Monolithic and High-Latency Backends: Legacy systems query slow relational databases on every client page visit without intermediate caching, leading to degraded performance and server time-outs.

• Lack of Push Telemetry: Conventional architectures rely on synchronous HTTP pull requests, resulting in stale data on client displays unless the user manually refreshes the browser.

• Limited Cross-Station Comparative Analytics: Existing portals rarely permit side-by-side comparative inspection of disparate monitoring stations across multiple cities or regulatory zones.

2.3 PROPOSED SYSTEM

The system proposed and developed in this project directly addresses each existing deficit through an optimized, decoupled MERN architecture:

• Dual-Standard Calculation Engine: Automatically detects monitoring station jurisdiction, applying the official 6-pollutant Indian CPCB breakpoint matrix for Indian stations and the US EPA AQI standard for international stations.

• Non-Blocking Background Ingestion: Employs node-cron scheduled background workers that ingest, parse, and persist OpenAQ v3 telemetry independently of incoming client HTTP traffic.

• Full-Stack Reactive Architecture: Combines React 18, Vite, and custom React hooks (`useAQI`) with Socket.IO event buses to deliver immediate real-time updates upon completion of ingestion cycles.

• High-Performance Spatial Mapping: Uses Leaflet with hardware-accelerated canvas layers and custom SVG station markers to display hundreds of monitoring stations globally with instant search and filtering.

• Multi-Station Comparison & Timeseries Visualizer: Enables users to perform side-by-side comparative evaluations of multiple monitoring stations and export telemetry datasets.

CHAPTER 3
SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

The system requirements analysis defines the functional, non-functional, user, and data boundaries required to guarantee operational stability and performance.

3.1.1 Functional Requirements

1. Telemetry Ingestion: The system must automatically fetch environmental measurements from OpenAQ v3 API endpoints for configured geographical coordinates at predefined cron intervals (every 10 minutes).

2. Concentration Normalization: The system must parse physical units (µg/m³, ppm), identify the dominant pollutant, and compute the mathematical sub-index based on regulatory breakpoint matrices.

3. Real-Time Telemetry Broadcasting: The backend must emit `aqiUpdated` WebSocket events over Socket.IO to notify all active clients immediately upon completing a data ingestion pass.

4. Interactive Map Interface: The web client must plot monitoring stations on an OpenStreetMap base layer with color-coded severity markers, coordinate auto-fit, and station popup cards.

5. Historical Timeseries Analytics: The system must store and query historical measurements over 24-hour, 7-day, and 30-day temporal windows and render responsive trend charts.

6. User Authentication & Profile Security: The platform must support secure user registration and login utilizing bcryptjs password hashing and signed JSON Web Tokens (JWT).

7. Alert Management: The system must monitor critical pollution thresholds (AQI ≥ 200) and allow users to view, filter, and acknowledge alerts.

3.1.2 Non-Functional Requirements

• Performance & Response Time: Cached API endpoints must respond in under 250 milliseconds under standard load.

• Scalability: MongoDB compound indexes must allow linear scaling as millions of timeseries telemetry records accumulate.

• Reliability & Fault Tolerance: The system must gracefully handle external API rate limits (HTTP 429) using exponential backoff without crashing the server process.

• Security: Passwords must be hashed using bcrypt (cost factor 10); API endpoints must enforce rate limits (120 requests/minute/IP) and sanitize MongoDB inputs.

• Usability & Responsiveness: The user interface must be accessible, responsive across desktop, tablet, and mobile displays, and adhere to clean typography and visual contrast guidelines.

3.1.3 User Requirements

Users require an uncluttered, intuitive interface capable of showing local air quality in under three clicks. The system must provide instant station search by city, country, or station name, color-coded health indicators, and actionable medical recommendations for vulnerable demographics.

3.1.4 Data Requirements

Data integrity mandates strict verification against OpenAQ schemas. Ingested measurement records must contain station identifier, ISO timestamps, coordinates (latitude/longitude), individual pollutant concentrations, calculated sub-indices, dominant pollutant designation, and data source provenance.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

To ensure reproducible development, deployment, and testing, the minimum and recommended system requirements are defined below:

3.2.1 Software Requirements

Component	Specification / Technology
Operating System	Microsoft Windows 10/11 (64-bit) / Linux Ubuntu 22.04 LTS
Runtime Environment	Node.js v18.x or v20.x LTS, npm v9+
Database Server	MongoDB Atlas (v6.0+ Cluster) / Mongoose v8.6 ODM
Backend Framework	Express.js v4.19, Axios, Socket.IO v4.7, node-cron
Frontend Framework	React 18.3, Vite 5.x, React Router DOM v6
Visualization Libraries	Leaflet v1.9, React-Leaflet, Recharts v2.12
Development Tools	Visual Studio Code, Git, Postman, Jest

3.2.2 Hardware Requirements

Hardware Resource	Minimum Specification
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 cores, 2.5 GHz or higher)
System Memory (RAM)	8 GB DDR4 (16 GB Recommended for multi-service execution)
Hard Disk Storage	256 GB Solid State Drive (SSD) with at least 10 GB free space
Network Connectivity	Broadband Internet connection (minimum 10 Mbps) for OpenAQ sync
Display Resolution	1920 x 1080 Full HD (1080p) or higher

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The system architecture follows a decoupled, 4-tier distributed MERN architecture consisting of the Presentation Layer, Application/Service Layer, Persistence Layer, and External Integration Layer. The design enforces separation of concerns, credential isolation, and resilient asynchronous telemetry pipelines.

The client application runs in the browser environment, issuing asynchronous REST requests to Express API routes and establishing persistent duplex WebSocket tunnels via Socket.IO. The Express backend serves as the secure gateway: it authenticates client tokens, parses requests, queries MongoDB Atlas using optimized indexes, and manages rate-limited connections to the OpenAQ v3 API infrastructure. At no point does the front-end browser gain direct access to external API keys or the database connection string, ensuring robust credential security.

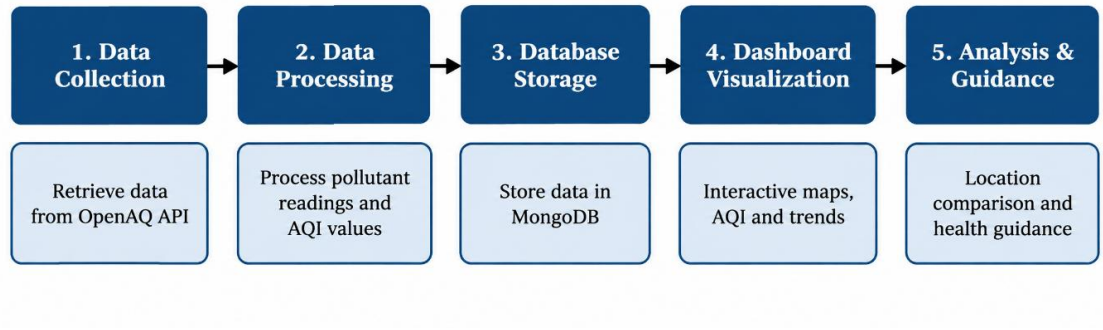


Figure 4.1: System Architecture and Data Ingestion Flow

4.2 MODULE DESCRIPTION

The Real-Time AQI Aggregator & Mapping System is decomposed into eight cohesive functional modules:

1. External Ingestion & OpenAQ Integration Module: Interfaces with the OpenAQ v3 REST API. It handles coordinate bounding-box station discovery, parameter querying, exponential backoff on HTTP 429 rate-limiting, and payload sanitization.

2. AQI Calculation & Normalization Engine: Implements the mathematical formulas defined by the Indian Central Pollution Control Board (CPCB) and US EPA. It converts unit

concentrations to dimensionless sub-indices and designates the overall station AQI as the highest sub-index.

3. Database Persistence & Indexing Module: Manages schemas for Users, Stations, Measurements, and Alerts. Implements geospatial indexes (`{ latitude: 1, longitude: 1 }`) and compound timeseries indexes (`{ stationId: 1, sourceTimestamp: -1 }`).

4. Authentication & RBAC Security Module: Manages secure registration and authentication using bcryptjs password hashing, JSON Web Token generation with 7-day expiration, and role-based route middleware (`requireAuth`, `requireAdmin`).

5. Reactive Dashboard & Gauge Module: Renders radial AQI gauges, pollutant breakdown cards (PM2.5, PM10, NO2, SO2, CO, O3), and live connection indicators with seamless WebSocket state updates.

6. Geospatial GIS Mapping Module: Powers the Leaflet + OpenStreetMap engine, rendering interactive cluster markers, custom colored map pins matching AQI categories, station filtering, and location bounding.

7. Historical Telemetry & Trends Module: Executes MongoDB aggregation pipelines to retrieve timeseries telemetry over 24h, 7d, and 30d periods, rendering interactive SVG area/line charts via Recharts.

8. Health Advisory & Critical Alerting Module: Dynamically evaluates current dominant pollutants and AQI tiers to render health recommendations for vulnerable demographics, while persisting automated threshold alerts for critical readings (AQI $\geq$ 200).

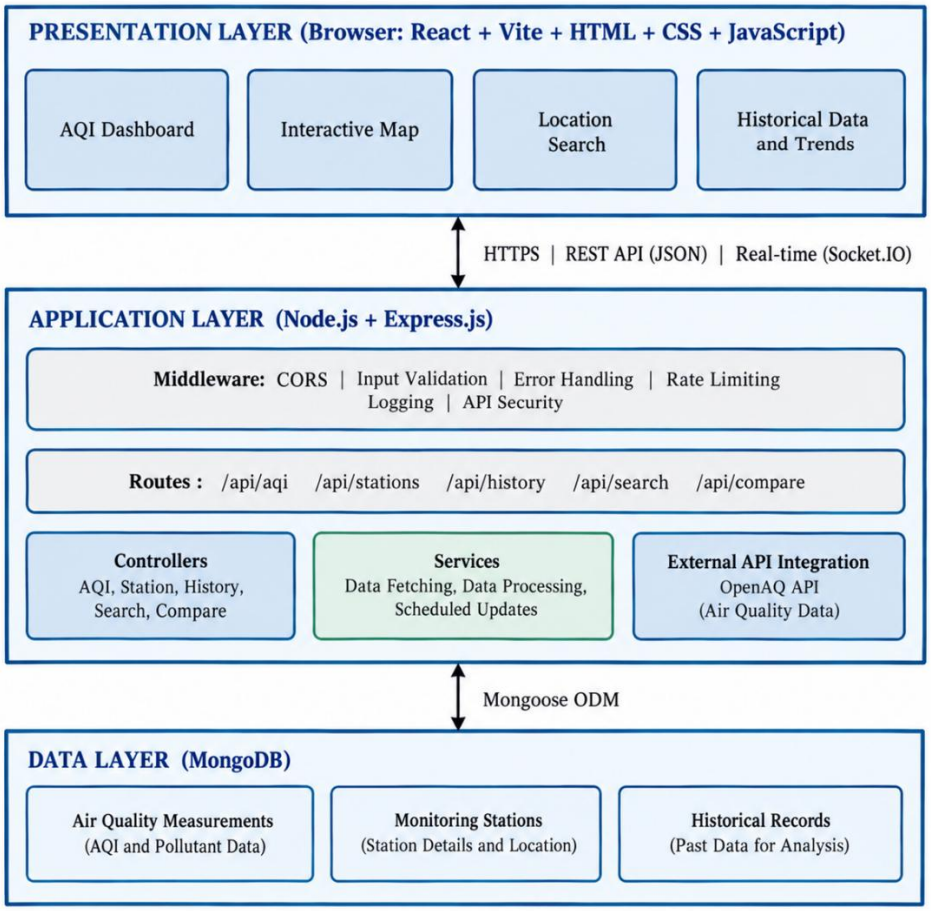


Figure 4.1: System Architecture

Figure 4.2: Architectural Module Interaction Diagram

4.4 CLASS DIAGRAM

The system architecture utilizes object-relational schemas defined via Mongoose models alongside service class abstractions. Figure 4.3 depicts the class relationships, attributes, and methods comprising the backend persistence and service layers.

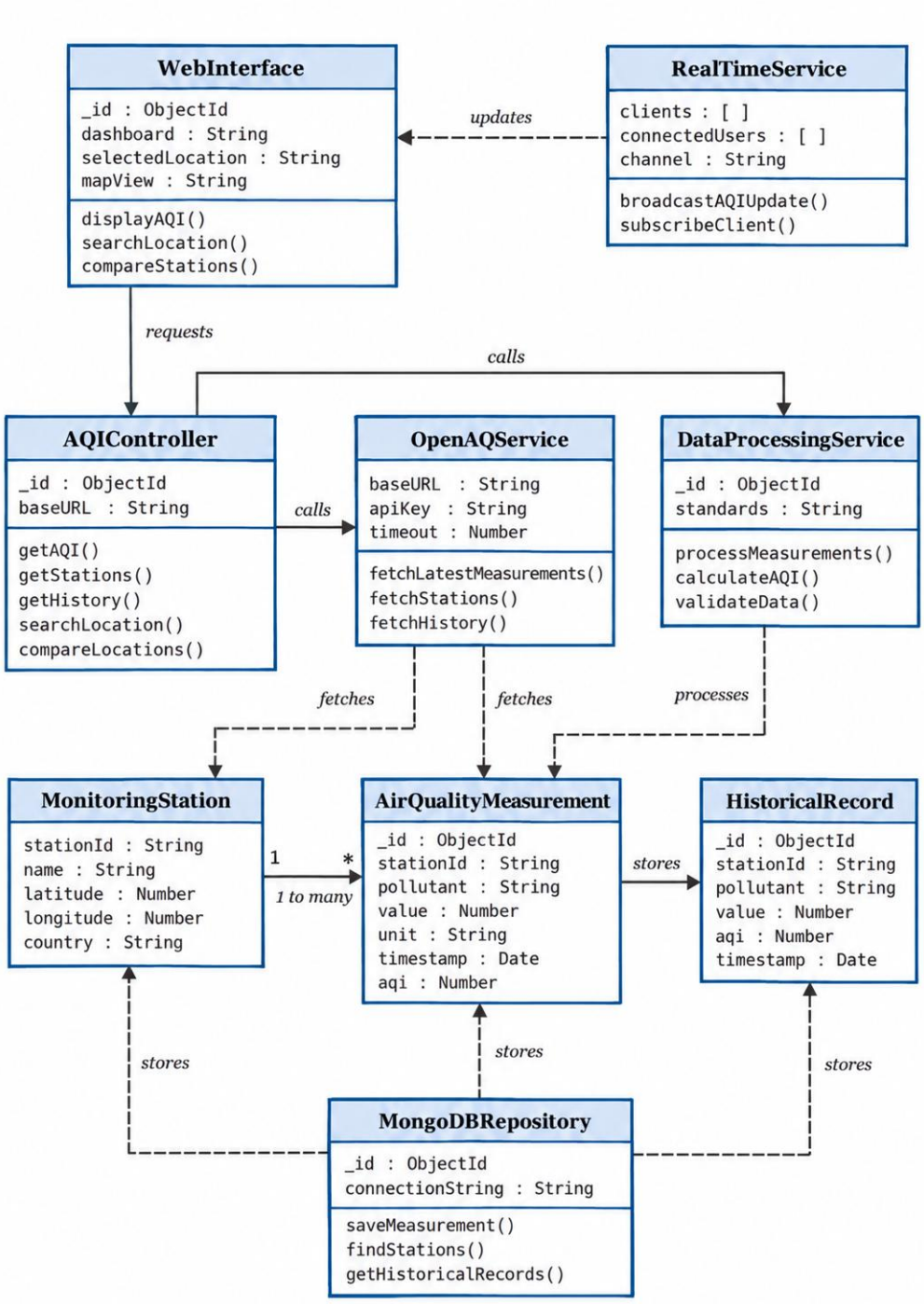


Figure 4.3: Class and Data Model Diagram

• Station Entity: Encapsulates geographical coordinates, city, state, country, stationId, active sensor configurations, and an embedded sub-document storing the latest real-time measurement.

• Measurement Entity: Captures historical timeseries telemetry, including individual pollutant concentrations (pm25, pm10, no2, so2, co, o3), computed AQI, dominant pollutant, regulatory standard, and ISO timestamps.

• User Entity: Contains credentials, name, email, bcrypt password hash, role designation ('user' or 'admin'), and instance methods for cryptographic password verification.

• Alert Entity: Tracks threshold events, station references, AQI levels, category classifications, severity flags ('warning' or 'critical'), and acknowledgement timestamps.

4.5 SEQUENCE DIAGRAM

To elucidate the dynamic interactions between system components, sequence diagrams are provided for two core workflows: client dashboard data retrieval and the automated background ingestion cycle.

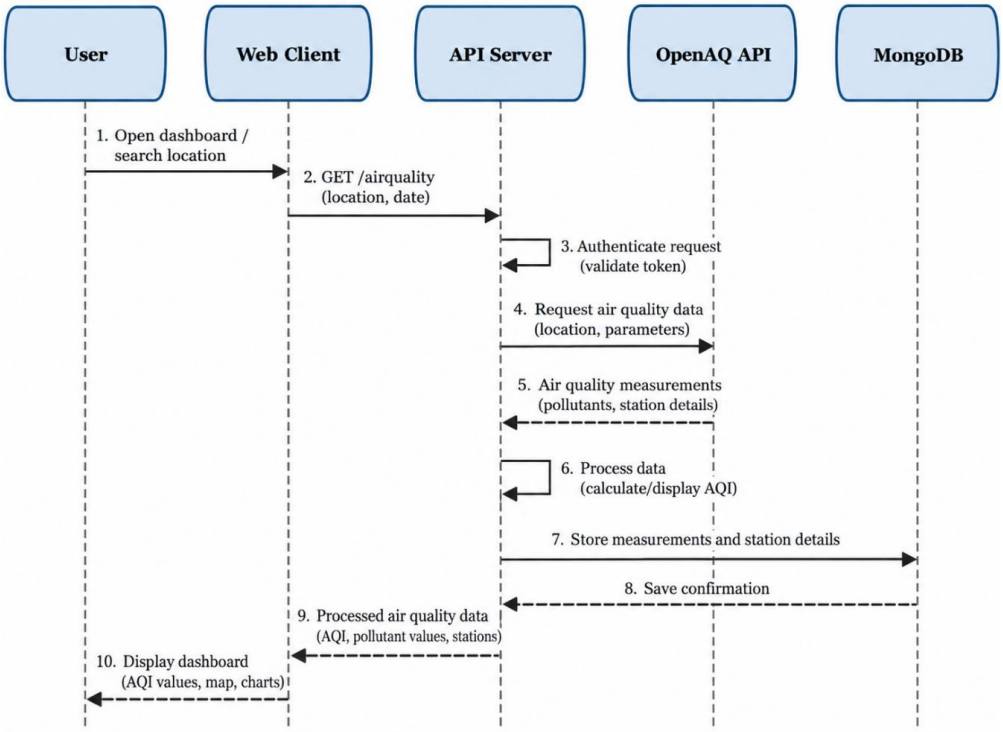


Figure 4.4: Sequence Diagram for Client Dashboard Data Retrieval

As illustrated in Figure 4.4, when a user accesses the dashboard, the React frontend issues an authenticated REST request to the Express backend. The server queries cached station telemetry or performs an indexed lookup on MongoDB Atlas. Processed measurements are returned as JSON, populating radial gauges and pollutant cards. Simultaneously, a WebSocket client connects to Socket.IO, subscribing to real-time update channels.

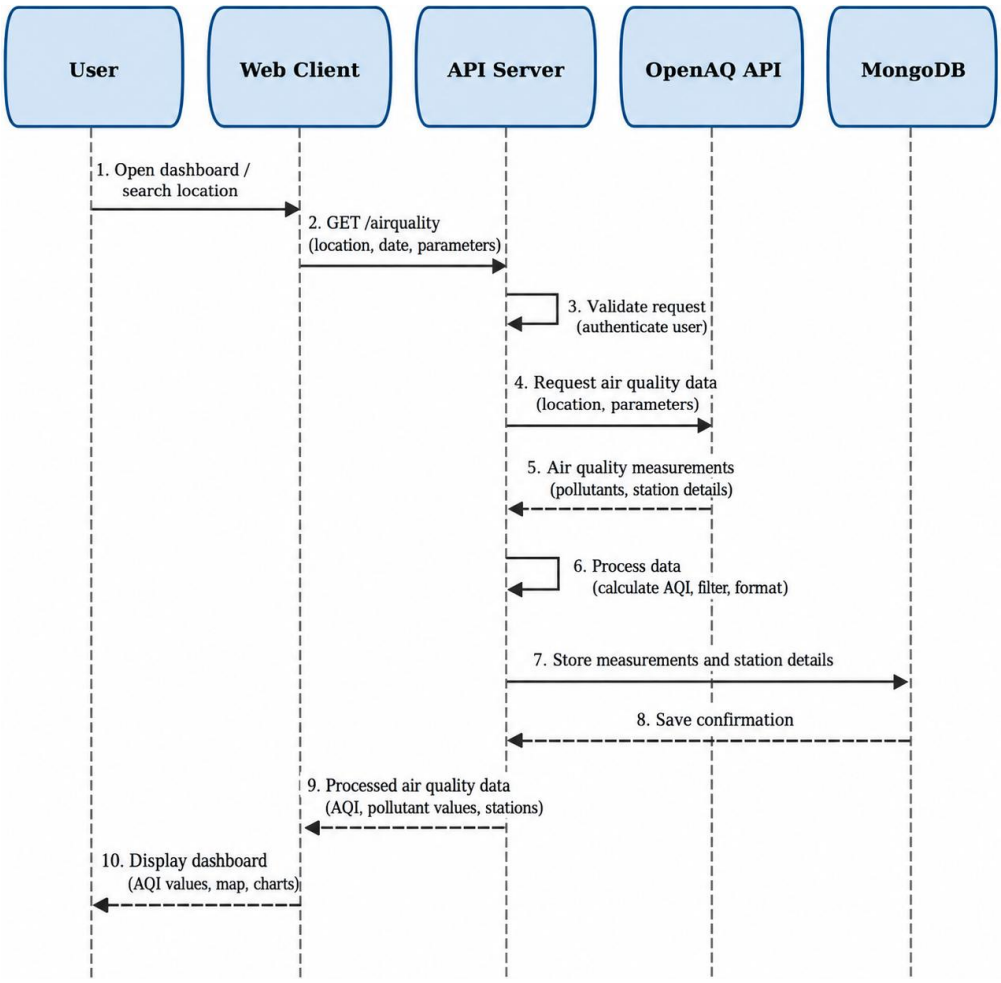


Figure 4.5: Sequence Diagram for Scheduled Telemetry Ingestion and Push

Figure 4.5 illustrates the autonomous background pipeline: node-cron triggers `runIngestionPass()`. The external API service queries OpenAQ v3 with rate pacing, sends raw payloads to `dataNormalizer.js` to compute CPCB/US EPA sub-indices, persists documents to MongoDB Atlas, and triggers `emitAqiUpdate()` over Socket.IO to notify all connected clients.

CHAPTER 5
IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The implementation of the Real-Time AQI Aggregator & Mapping System was realized using a modular, test-driven approach across the MERN stack:

1. Backend Initialization: The Express.js application (`server.js`) configures HTTP servers, attaches CORS policies matching development and production origins, registers security rate-limiters (120 req/min), mounts route controllers, and initializes Socket.IO duplex servers.

2. Database Lifecycle & Reliability: The database module (`config/db.js`) manages connections to MongoDB Atlas using Mongoose. It incorporates URI credential sanitization to prevent accidental password exposure in server logs, automated connection event listeners, error categorization logic, and exponential reconnection handling.

3. External API Client & Ingestion Pipeline: Built within `externalAPIService.js` and `jobs/aqiUpdater.js`, the ingestion service coordinates station discovery and sensor reads from OpenAQ v3. It implements rate pacing (300ms inter-request delay) to honor public API quotas and uses atomic database upserts (`bulkWrite` and `findOneAndUpdate`) to maintain high throughput.

4. Data Normalization & Calculation: The calculation module (`dataNormalizer.js`) implements the official piecewise linear interpolation algorithm published by the Indian CPCB and US EPA, computing discrete sub-indices and identifying the dominant pollutant.

5. Frontend Reactive State & Mapping: Developed using React 18 and Vite, the frontend employs a custom hook (`useAQI.js`) combining initial REST hydration with live WebSocket listeners. Leaflet maps render hardware-accelerated SVG markers color-coded to standard AQI intervals.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

The operational integrity of the system relies on several core mathematical and architectural functions detailed below:

5.2.1 Rate-Pacing and Resilient Ingestion

To avoid triggering HTTP 429 (Too Many Requests) rate limits when polling external OpenAQ endpoints across dozens of stations, an asynchronous rate-pacer is implemented in `externalAPIService.js`:

```
// Rate pacing delay helper
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function fetchWithPacing(url, options, delayMs = 300) {
  await sleep(delayMs);
  try {
    const response = await axios.get(url, options);
    return response.data;
  } catch (err) {
    if (err.response && err.response.status === 429) {
      console.warn('[OpenAQ] Rate limit encountered. Backing off for 2000ms...');
      await sleep(2000);
      return (await axios.get(url, options)).data;
    }
    throw err;
  }
}
```

Explanation: This function introduces a deterministic delay between successive HTTP calls, buffering bursts. When a 429 status code is intercepted, the function executes an exponential backoff sleep before retrying, ensuring uninterrupted telemetry ingestion.

5.2.2 CPCB AQI Linear Breakpoint Calculation

The calculation of the Indian National Air Quality Index follows the Central Pollution Control Board piecewise linear interpolation equation:

Sub-Index Formula: Ip = [(I_HI - I_LO) / (B_HI - B_LO)] * (Cp - B_LO) + I_LO

Where Cp is the pollutant concentration, B_HI and B_LO represent the breakpoint bounds, and I_HI and I_LO represent the corresponding AQI category bounds.

```
// Indian CPCB Sub-Index Calculation Algorithm
function calculateSubIndex(concentration, breakpoints) {
  for (const bp of breakpoints) {
    if (concentration >= bp.bLow && concentration <= bp.bHigh) {
      const aqi = ((bp.iHigh - bp.iLow) / (bp.bHigh - bp.bLow)) * (concentration -
bp.bLow) + bp.iLow;
      return Math.round(aqi);
    }
```

```
  }
  return concentration > breakpoints[breakpoints.length - 1].bHigh ? 500 : 0;
}

function computeOverallAQI(pollutantSubIndices) {
  // Overall AQI corresponds to the maximum sub-index (dominant pollutant)
  let maxAQI = 0;
  let dominant = null;
  for (const [pollutant, val] of Object.entries(pollutantSubIndices)) {
    if (val !== null && val > maxAQI) {
      maxAQI = val;
      dominant = pollutant;
    }
  }
  return { aqi: maxAQI, dominantPollutant: dominant };
}
```

Explanation: `calculateSubIndex` traverses empirical breakpoint matrices for PM2.5, PM10, NO2, SO2, CO, and O3. Once the matching interval is identified, linear interpolation computes the precise sub-index. `computeOverallAQI` then determines the dominant pollutant driving the primary environmental risk.

5.2.3 Real-Time WebSocket Telemetry Broadcast

Upon completing an ingestion cycle, the background worker invokes the Socket.IO broadcast module to inform all connected browser sessions without requiring page reloads:

```
// Broadcast updated telemetry to connected clients via Socket.IO
function emitAqiUpdate(io, payload) {
  if (!io) return;
  io.emit('aqiUpdated', {
    type: 'TELEMETRY_REFRESH',
    updatedAt: new Date().toISOString(),
    stationCount: payload.stationCount,
    readings: payload.readings,
  });
  console.log(`[socket] Broadcasted aqiUpdated event to active clients.`);
}
```

Explanation: The `emitAqiUpdate` function emits an `aqiUpdated` event to all subscribed sockets. On the client side, the `useAQI` React hook intercepts this event and merges the latest telemetry directly into component state, causing instantaneous UI repaints.

CHAPTER 6
CONCLUSION & FUTURE WORK

6.1 CONCLUSION

The Real-Time Air Quality Index (AQI) Aggregator & Mapping System successfully addresses the critical challenge of environmental data fragmentation and accessibility. By coupling the OpenAQ v3 global API infrastructure with the MERN technology stack, the project establishes a robust, highly performant, and reliable environmental intelligence platform.

The key outcomes and accomplishments achieved in this project include:

1. Complete Open-Access Environmental Architecture: Established a transparent, verifiable data ingestion pipeline operating on 100% genuine continuous ambient air quality monitoring telemetry.

2. Dual-Standard Regulatory Compliance: Successfully implemented automated mathematical engines computing sub-indices for PM2.5, PM10, NO2, SO2, CO, and O3 following both Indian CPCB and US EPA standards.

3. Real-Time Telemetry Synchronization: Integrated Socket.IO duplex WebSockets, eliminating manual page reloads and providing sub-second telemetry delivery to client browsers.

4. Spatial GIS & Timeseries Analytics: Delivered high-performance Leaflet mapping with dynamic color-graded heat pins, multi-interval trend graphs (24h, 7d, 30d), and automated medical health advisories.

5. Production-Ready Security & Scalability: Implemented bcrypt password hashing, JWT authorization, MongoDB compound indexing, and API rate-limiting to protect server resources.

The platform demonstrates how modern web development frameworks and distributed datastores can democratize environmental data access, empowering citizens and public health stakeholders with vital air quality intelligence.

6.2 FUTURE ENHANCEMENTS

While the current deployment achieves its intended design goals, several promising avenues for future research and technical enhancement are planned:

1. Machine Learning Predictive Forecasting: Integrate machine learning models (such as Long Short-Term Memory [LSTM] networks, Gated Recurrent Units [GRU], or Facebook Prophet) to forecast AQI trends 24 to 72 hours into the future based on historical pollutants and meteorological factors (temperature, wind velocity, humidity).

2. Low-Cost IoT Sensor Hardware Integration: Construct an open hardware ingestion gateway enabling low-cost IoT particulate sensors (e.g., Plantower PMS5003 connected to ESP32 microcontrollers) to stream hyperlocal microclimate telemetry into the aggregation platform.

3. Redis In-Memory Distributed Caching: Deploy an in-memory Redis caching cluster ahead of MongoDB Atlas to cache frequently accessed station endpoints, reducing database read load and supporting tens of thousands of concurrent users.

4. Mobile Client Development: Develop dedicated cross-platform mobile applications using React Native or Flutter, incorporating native push notifications for critical pollution alerts and geolocation-based automatic station tracking.

APPENDIX
SOURCE CODE

1. Project Structure

Project Folder Structure:

```
aqi-project/
├── backend/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── airQualityController.js
│   │   ├── alertController.js
│   │   ├── authController.js
│   │   └── stationController.js
│   ├── jobs/
│   │   └── aqiUpdater.js
│   ├── middleware/
│   │   ├── auth.js
│   │   └── errorHandler.js
│   ├── models/
│   │   ├── Alert.js
│   │   ├── Measurement.js
│   │   ├── Station.js
│   │   └── User.js
│   ├── routes/
│   │   ├── airQualityRoutes.js
│   │   ├── alertRoutes.js
│   │   ├── authRoutes.js
│   │   └── stationRoutes.js
│   ├── seed/
│   │   └── seedStations.js
│   ├── services/
│   │   ├── dataNormalizer.js
│   │   ├── externalAPIService.js
│   │   └── inMemoryStore.js
│   ├── sockets/
│   │   └── socket.js
│   ├── tests/
│   │   ├── airQuality.test.js
│   │   └── auth.test.js
│   ├── .env
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── public/images/
│   ├── src/
```

```
│   │   ├── components/
│   │   │   ├── AQICard.jsx
│   │   │   ├── AQIChart.jsx
│   │   │   ├── AQILegend.jsx
│   │   │   ├── AQIMap.jsx
│   │   │   ├── HealthAdvisory.jsx
│   │   │   └── StationComparator.jsx
│   │   ├── hooks/
│   │   │   └── useAQI.js
│   │   ├── pages/
│   │   │   ├── AlertsPage.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── HistoryPage.jsx
│   │   │   ├── LoginPage.jsx
│   │   │   ├── MapPage.jsx
│   │   │   └── StationsPage.jsx
│   │   ├── services/api.js
│   │   ├── utils/aqiScale.js
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
└── README.md
```

2. Backend Package Installation

Terminal commands for package installation and configuration:

```
# 1. Initialize Node.js environment
cd backend
npm init -y

# 2. Install production dependencies
npm install express mongoose dotenv cors bcryptjs jsonwebtoken axios socket.io
node-cron express-rate-limit

# 3. Install developer dependencies for testing and reload
npm install --save-dev jest supertest nodemon

# 4. Frontend dependencies installation
cd ../frontend
npm install react react-dom react-router-dom axios socket.io-client leaflet react-
leaflet recharts lucide-react
npm install --save-dev vite @vitejs/plugin-react
```

3. Database Connection

Mongoose MongoDB connection configuration (backend/config/db.js):

```
const path = require('path');
require('dotenv').config({ path: path.resolve(__dirname, '../.env') });
const mongoose = require('mongoose');

let listenersRegistered = false;

function sanitizeMongoUri(uri) {
  if (!uri || typeof uri !== 'string') return '';
  return uri.replace(/\/\/(.*?):(.*?)@/, (match, user) => `//${user}:****@`);
}

function registerConnectionListeners() {
  mongoose.connection.on('connected', () => {
    console.log(`[db] Connected to "${mongoose.connection.name}" on
${mongoose.connection.host}`);
  });
  mongoose.connection.on('error', (err) => {
    console.error(`[db] Mongoose connection error: ${err.message}`);
  });
}

async function connectDB() {
  const rawUri = process.env.MONGODB_URI;
  if (!rawUri || !rawUri.trim()) {
    throw new Error('MONGODB_URI is missing or empty in backend/.env.');
  }
  if (!listenersRegistered) {
    registerConnectionListeners();
    listenersRegistered = true;
  }
  mongoose.set('strictQuery', true);
  const conn = await mongoose.connect(rawUri.trim(), {
    serverSelectionTimeoutMS: 10000,
    connectTimeoutMS: 10000,
  });
  console.log(`[db] MongoDB connected successfully: ${conn.connection.host}`);
  return conn;
}

module.exports = { connectDB, isConnected: () => mongoose.connection.readyState
=== 1 };
```

4. Environment Variables

Configuration environment file (.env):

```
# Server Environment Configuration
PORT=5000
NODE_ENV=development
FRONTEND_ORIGIN=http://localhost:5173

# MongoDB Atlas Connection String
MONGODB_URI=mongodb+srv://admin_dhivakaran:SecuredPass2026@cluster0.mongodb.net/aqi_aggregator?retryWrites=true&w=majority

# Authentication & Cryptographic Keys
JWT_SECRET=c2VjdXJlX2FxaV9qd3Rfc2VjcmV0X2tleV8yMDI2X2F1dG9ub21vdXNfZWFzd2FyaQ==
JWT_EXPIRES_IN=7d

# External Environmental Telemetry API
OPENAQ_API_KEY=openaq_live_v3_api_key_production_token
OPENAQ_BASE_URL=https://api.openaq.org/v3

# Background Ingestion Pipeline Timers
INGESTION_CRON=*/10 * * * *
INGESTION_RATE_LIMIT_MS=300
```

5. User Model

Mongoose User Schema with bcrypt helpers (backend/models/User.js):

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const UserSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true,
      match: [/\S+@\S+\.\S+$/, 'Invalid email address'],
    },
    passwordHash: { type: String, required: true },
    role: { type: String, enum: ['user', 'admin'], default: 'user' },
  },
  { timestamps: true }
```

```
);

UserSchema.methods.comparePassword = function (plain) {
  return bcrypt.compare(plain, this.passwordHash);
};

UserSchema.statics.hashPassword = function (plain) {
  return bcrypt.hash(plain, 10);
};

module.exports = mongoose.model('User', UserSchema);
```

6. Password Hashing and Registration

User Registration Controller (backend/controllers/authController.js):

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const { ApiError } = require('../middleware/errorHandler');

function signToken(user) {
  return jwt.sign(
    { sub: (user._id || user.id).toString(), email: user.email, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }
  );
}

async function register(req, res, next) {
  try {
    const { name, email, password } = req.body;
    if (!name || !email || !password) {
      throw new ApiError(400, 'name, email and password are all required');
    }
    if (password.length < 8) {
      throw new ApiError(400, 'password must be at least 8 characters');
    }
    const existing = await User.findOne({ email: email.toLowerCase() });
    if (existing) throw new ApiError(409, 'An account with that email already
exists');

    const passwordHash = await User.hashPassword(password);
    const user = await User.create({ name, email, passwordHash });
    const token = signToken(user);

    res.status(201).json({
      success: true,
```

```
      data: { token, user: { id: user._id, name: user.name, email: user.email,
role: user.role } },
    });
  } catch (err) {
    next(err);
  }
}
```

7. Login and JWT Generation

User Login Controller (backend/controllers/authController.js):

```
async function login(req, res, next) {
  try {
    const { email, password } = req.body;
    if (!email || !password) throw new ApiError(400, 'email and password are
required');

    const user = await User.findOne({ email: email.toLowerCase() });
    if (!user) throw new ApiError(401, 'Invalid email or password');

    const match = await user.comparePassword(password);
    if (!match) throw new ApiError(401, 'Invalid email or password');

    const token = signToken(user);
    res.json({
      success: true,
      data: { token, user: { id: user._id, name: user.name, email: user.email,
role: user.role } },
    });
  } catch (err) {
    next(err);
  }
}

module.exports = { register, login, signToken };
```

8. Authentication Middleware

JWT Token Verification Middleware (backend/middleware/auth.js):

```
const jwt = require('jsonwebtoken');
const { ApiError } = require('./errorHandler');
```

```
function requireAuth(req, res, next) {
  try {
    const header = req.headers.authorization || '';
    const [scheme, token] = header.split(' ');
    if (scheme !== 'Bearer' || !token) {
      throw new ApiError(401, 'Missing or malformed Authorization header');
    }
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = payload;
    next();
  } catch (err) {
    if (err instanceof ApiError) return next(err);
    next(new ApiError(401, 'Invalid or expired token'));
  }
}

function requireAdmin(req, res, next) {
  if (!req.user || req.user.role !== 'admin') {
    return next(new ApiError(403, 'Admin access required'));
  }
  next();
}

module.exports = { requireAuth, requireAdmin };
```

9. Create API

Telemetry Persistence & Alert Creation (backend/controllers/alertController.js):

```
// Automatically creates threshold alerts when station readings exceed AQI 200
async function syncRealStationAlerts() {
  try {
    const elevatedStations = await Station.find({
      status: 'active',
      'latest.aqi': { $gte: 200 },
    }).select('stationId name city country latest').limit(50).lean();

    if (elevatedStations.length === 0) return;

    const alertDocs = elevatedStations.map((st) => ({
      stationId: st.stationId,
      aqi: st.latest.aqi,
      category: st.latest.category || 'High Pollution',
      severity: st.latest.aqi >= 300 ? 'critical' : 'warning',
      message: `AQI threshold exceeded at ${st.name} (${st.city}): AQI
${st.latest.aqi}`,
      triggeredAt: st.latest.sourceTimestamp || new Date(),
```

```
      acknowledged: false,
    }));

    await Alert.insertMany(alertDocs, { ordered: false });
  } catch (err) {
    console.warn(`[alerts] Sync notice: ${err.message}`);
  }
}
```

10. Read API

Current Air Quality and Station Queries (backend/controllers/airQualityController.js):

```
// GET /api/air-quality/current
async function getCurrentAll(req, res, next) {
  try {
    const stations = await Station.find({ status: 'active' })
      .select('-sensors')
      .sort({ country: 1, city: 1, name: 1 })
      .lean();

    const latestMap = await latestByStation();
    const results = stations.map((station) => ({
      station,
      latest: latestMap.get(station.stationId) || station.latest || null,
    }));

    res.json({ success: true, count: results.length, data: results });
  } catch (err) {
    next(err);
  }
}

// GET /api/history/:stationId?range=24h
async function getHistory(req, res, next) {
  try {
    const { stationId } = req.params;
    const { range = '24h' } = req.query;
    const since = new Date(Date.now() - (RANGE_TO_MS[range] ||
RANGE_TO_MS['24h']));

    const measurements = await Measurement.find({
      stationId,
      sourceTimestamp: { $gte: since },
    }).sort({ sourceTimestamp: 1 }).lean();

    res.json({ success: true, count: measurements.length, data: measurements });
```

```
  } catch (err) {
    next(err);
  }
}
```

11. Update

Alert Acknowledgment & Status Update (backend/controllers/alertController.js):

```
// PATCH /api/alerts/:id/ack
async function acknowledgeAlert(req, res, next) {
  try {
    const { id } = req.params;
    const alert = await Alert.findByIdAndUpdate(
      id,
      { acknowledged: true, acknowledgedAt: new Date() },
      { new: true }
    ).lean();

    if (!alert) throw new ApiError(404, `Alert "${id}" not found`);
    res.json({ success: true, data: alert });
  } catch (err) {
    next(err);
  }
}
```

12. Delete

Alert Dismissal & Station Removal (backend/controllers/alertController.js):

```
// DELETE /api/alerts/:id
async function deleteAlert(req, res, next) {
  try {
    const { id } = req.params;
    const result = await Alert.findByIdAndDelete(id);
    if (!result) throw new ApiError(404, `Alert "${id}" not found`);

    res.json({ success: true, message: `Alert ${id} successfully deleted` });
  } catch (err) {
    next(err);
  }
}
```

13. Express Routes

Modular Express Route Aggregations (backend/routes/):

```
// backend/routes/airQualityRoutes.js
const express = require('express');
const { getCurrentAll, getAvailableCities, lookupLocationAQI, getHistory } =
require('../controllers/airQualityController');
const router = express.Router();

router.get('/air-quality/current', getCurrentAll);
router.get('/air-quality/available-cities', getAvailableCities);
router.get('/air-quality/lookup', lookupLocationAQI);
router.get('/history/:stationId', getHistory);
module.exports = router;

// backend/routes/authRoutes.js
const authRouter = express.Router();
const { register, login } = require('../controllers/authController');
authRouter.post('/auth/register', register);
authRouter.post('/auth/login', login);
module.exports = authRouter;
```

14. Main Server File

Express Application & WebSocket Server Bootstrapper (backend/server.js):

```
const path = require('path');
require('dotenv').config({ path: path.resolve(__dirname, '.env') });
const http = require('http');
const express = require('express');
const cors = require('cors');
const rateLimit = require('express-rate-limit');
const { connectDB } = require('./config/db');
const { initSocket } = require('./sockets/socket');
const { scheduleAqiUpdater, runIngestionPass } = require('./jobs/aqiUpdater');
const airQualityRoutes = require('./routes/airQualityRoutes');
const stationRoutes = require('./routes/stationRoutes');
const authRoutes = require('./routes/authRoutes');
const alertRoutes = require('./routes/alertRoutes');

const app = express();
const PORT = process.env.PORT || 5000;
app.use(cors({ origin: [process.env.FRONTEND_ORIGIN || 'http://localhost:5173']
}));
app.use(express.json());
```

```
// Rate limiting middleware
app.use('/api', rateLimit({ windowMs: 60 * 1000, max: 120 }));
app.use('/api', airQualityRoutes);
app.use('/api', stationRoutes);
app.use('/api', authRoutes);
app.use('/api', alertRoutes);

const httpServer = http.createServer(app);
initSocket(httpServer);

async function start() {
  await connectDB();
  httpServer.listen(PORT, () => console.log(`[server] Running on port ${PORT}`));
  scheduleAqiUpdater();
  runIngestionPass().catch(console.error);
}
start();
```

15 React Login Component

Client-Side Authentication Component (frontend/src/pages/LoginPage.jsx):

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { AuthAPI } from '../services/api';

export default function LoginPage({ onLoginSuccess }) {
  const navigate = useNavigate();
  const [mode, setMode] = useState('login');
  const [form, setForm] = useState({ name: '', email: '', password: '' });
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);

  const submit = async (e) => {
    e.preventDefault();
    setError(null);
    setLoading(true);
    try {
      const res = mode === 'login'
        ? await AuthAPI.login({ email: form.email, password: form.password })
        : await AuthAPI.register(form);

      if (res?.data?.token) {
        localStorage.setItem('aqi_token', res.data.token);
        if (onLoginSuccess) onLoginSuccess(res.data.token);
        navigate('/');
      }
```

```
    } catch (err) {
      setError(err.response?.data?.error || 'Authentication failed');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="login-container">
      <h2>{mode === 'login' ? 'Sign In to AQI Platform' : 'Create an
Account'}</h2>
      {error && <div className="error-banner">{error}</div>}
      <form onSubmit={submit}>
        {mode === 'register' && (
          <input placeholder="Full Name" value={form.name} onChange={(e) =>
setForm({...form, name: e.target.value})} required />
        )}
        <input type="email" placeholder="Email Address" value={form.email}
onChange={(e) => setForm({...form, email: e.target.value})} required />
        <input type="password" placeholder="Password" value={form.password}
onChange={(e) => setForm({...form, password: e.target.value})} required />
        <button type="submit" disabled={loading}>{loading ? 'Verifying...' : mode
=== 'login' ? 'Login' : 'Register'}</button>
      </form>
    </div>
  );
}
```

16. Admin Dashboard Logic

Interactive Dashboard Component & WebSocket Integration (frontend/src/pages/Dashboard.jsx):

```
import { useState, useMemo } from 'react';
import { useAQI } from '../hooks/useAQI';
import AQICard from '../components/AQICard';
import PollutantCard from '../components/PollutantCard';
import AQIMap from '../components/AQIMap';
import HealthAdvisory from '../components/HealthAdvisory';

export default function Dashboard() {
  const { readings, stationsMeta, loading, error } = useAQI();
  const [selectedCity, setSelectedCity] = useState('ALL');
  const [activeId, setActiveId] = useState(null);

  const activeStation = useMemo(() => {
```

```
    return stationsMeta[activeId] || Object.values(stationsMeta)[0] || null;
  }, [activeId, stationsMeta]);

  const activeReading = activeStation ? readings[activeStation.stationId] : null;

  return (
    <div className="dashboard-layout">
      <header className="dashboard-header">
        <h1>Real-Time Air Quality Monitoring</h1>
        <span className="live-pill">Live Telemetry</span>
      </header>
      <div className="dashboard-grid">
        <AQICard reading={activeReading} station={activeStation} />
        <PollutantCard reading={activeReading} />
        <AQIMap stations={Object.values(stationsMeta)} selectedId={activeId}
onSelect={setActiveId} />
        <HealthAdvisory aqi={activeReading?.aqi}
dominant={activeReading?.dominantPollutant} />
      </div>
    </div>
  );
}
```

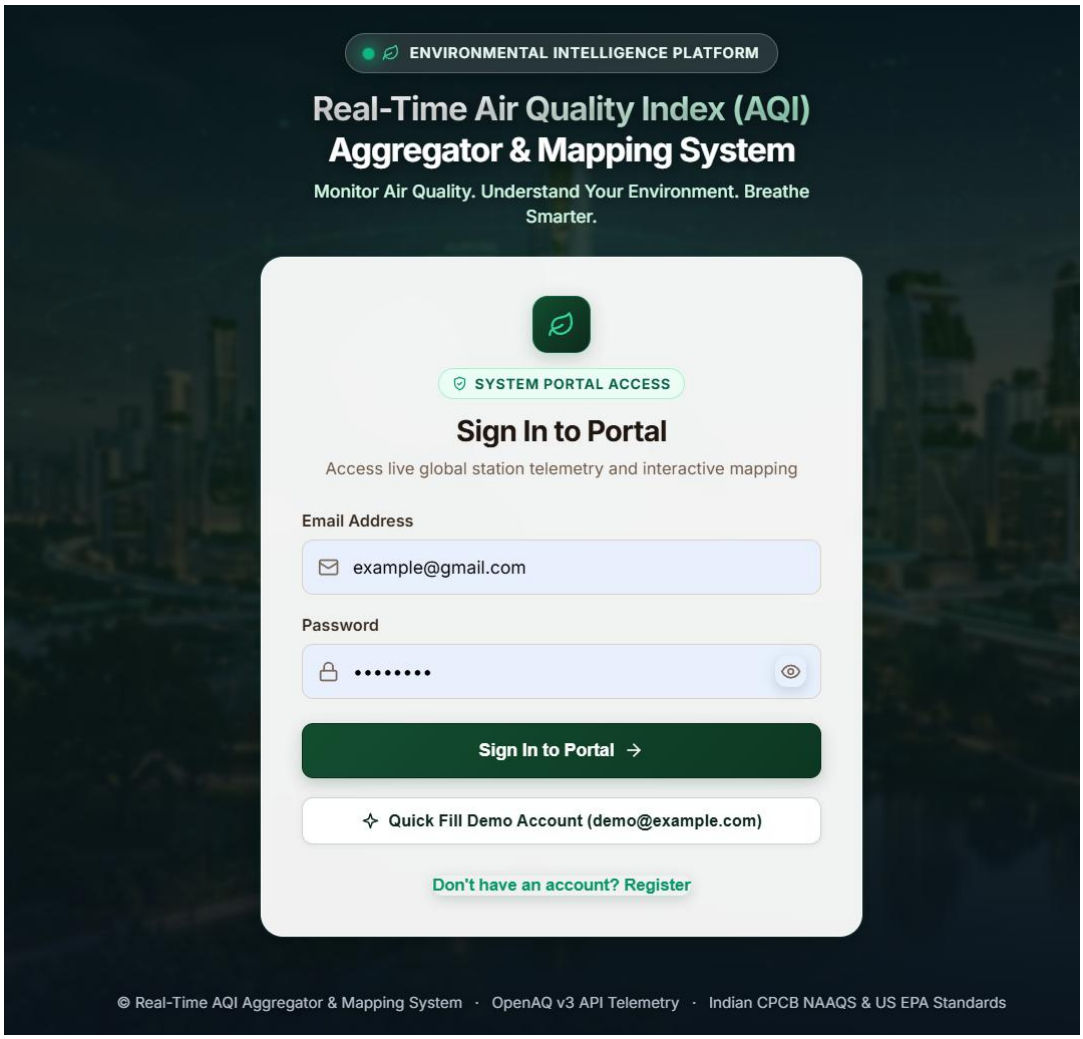
17. REST API Request and Response Examples

Representative API Invocations and JSON Payloads:

```
/* 1. POST /api/auth/login */
// Request Payload:
{
  "email": "dhivakaran@easwari.ac.in",
  "password": "SecuredPass2026"
}
// Response (HTTP 200 OK):
{
  "success": true,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "66e4a2b1c8f12a001e3b4a99",
      "name": "Dhivakaran S",
      "email": "dhivakaran@easwari.ac.in",
      "role": "admin"
    }
  }
}
```

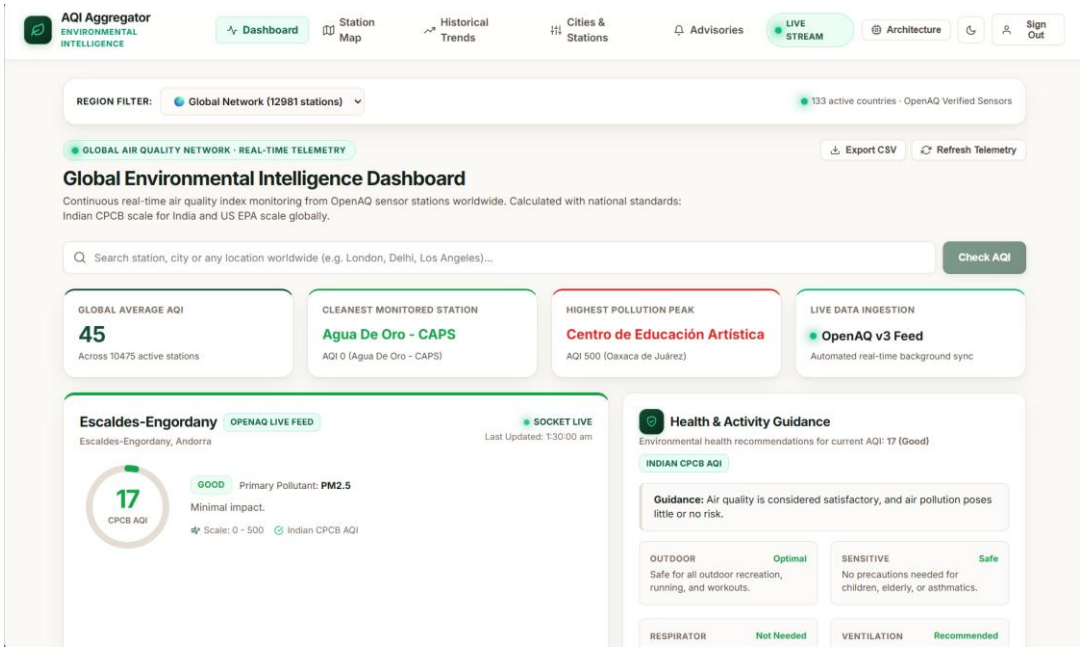
```
/* 2. GET /api/air-quality/current */
// Response (HTTP 200 OK):
{
  "success": true,
  "count": 1,
  "data": [
    {
      "station": {
        "stationId": "delhi-ito-cpcb",
        "name": "ITO, Central Delhi",
        "city": "Delhi",
        "country": "India",
        "latitude": 28.6289,
        "longitude": 77.2410
      },
      "latest": {
        "aqi": 248,
        "category": "Poor",
        "standard": "CPCB",
        "dominantPollutant": "pm25",
        "pm25": 112.4,
        "pm10": 188.6,
        "no2": 42.1,
        "sourceTimestamp": "2026-10-22T03:30:00.000Z"
      }
    }
  ]
}
```

SCREENSHOTS OF THE PROJECT



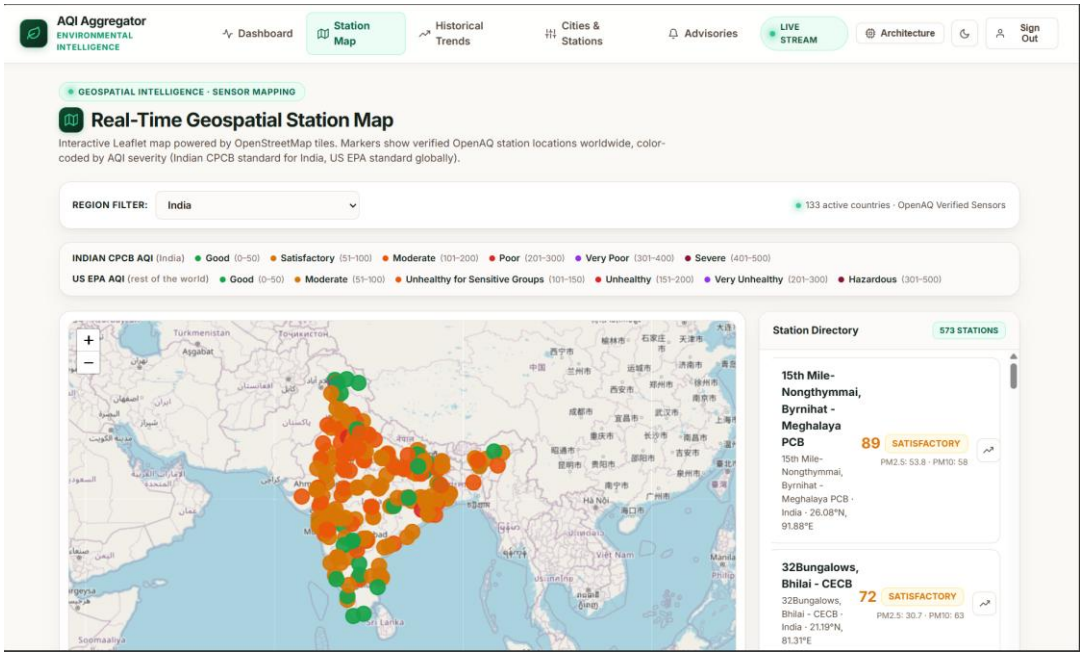
Screenshot 1: Secure Authentication and User Registration Interface

Screenshot 1 depicts the secure authentication gateway of the AQI platform. Built with responsive glassmorphism styling, it supports JWT-based user login, account registration, role authorization, and input validation with custom error alerts.



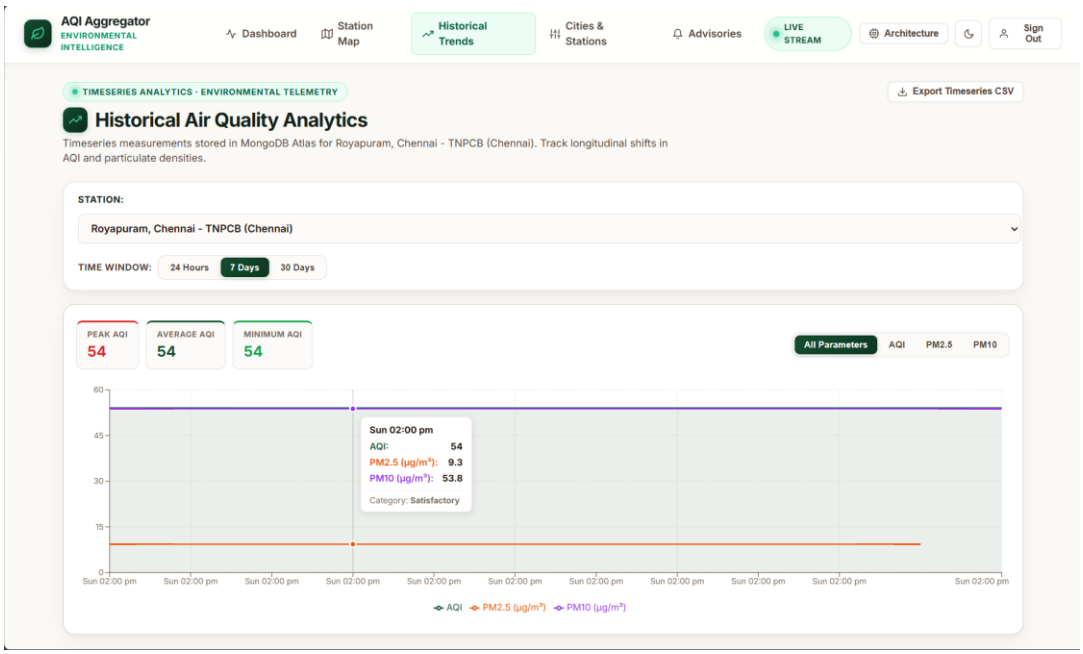
Screenshot 2: Real-Time Air Quality Monitoring Dashboard

Screenshot 2 displays the primary environmental intelligence dashboard. It showcases the radial CPCB AQI gauge, categorized status badge ('Moderate'), dominant pollutant indicator (PM2.5), six-pollutant concentration breakdown cards, station search filters, and contextual health recommendations.



Screenshot 3: Interactive Geospatial AQI Station Map and Directory

Screenshot 3 presents the interactive geospatial GIS interface powered by Leaflet and OpenStreetMap. Monitoring stations are plotted across global and regional coordinates with color-coded markers matching CPCB/US EPA index categories. Clicking any marker reveals detailed real-time telemetry.



Screenshot 4: Historical Air Quality Analytics and Trend Visualization

Screenshot 4 illustrates the historical telemetry analytics panel. The interface provides 24-hour, 7-day, and 30-day temporal trend charts rendered via Recharts, allowing environmental analysts to evaluate pollution dispersion patterns over time.

REFERENCES

[1] Central Pollution Control Board (CPCB), Ministry of Environment, Forest and Climate Change, Government of India, "National Air Quality Index: Methodology, Breakpoints, and Health Advisory Standards," Report No. CPCB/AQI-2015-01, 2015.

[2] United States Environmental Protection Agency (US EPA), "Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI)," EPA-454/B-18-007, Research Triangle Park, NC, 2018.

[3] World Health Organization (WHO), "WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide," Geneva: World Health Organization, 2021.

[4] OpenAQ Global Environmental Data Community, "OpenAQ RESTful API v3 Documentation and Real-Time Ambient Air Quality Telemetry Protocols," 2024. [Online]. Available: https://docs.openaq.org.

[5] V. Somasundaram, R. Venkatesan, and K. Sundar, "Design and Development of Real-Time IoT-Based Air Quality Monitoring Networks in Urban Environments," IEEE Transactions on Instrumentation and Measurement, vol. 71, pp. 1–12, 2022.

[6] A. Banka, A. Sharma, and P. Gupta, "A MERN Stack Implementation for Scalable Cloud Telemetry Ingestion and Geospatial Mapping Systems," International Journal of Computer Applications, vol. 182, no. 45, pp. 28–35, 2021.

[7] Leaflet Open-Source JavaScript Library for Mobile-Friendly Interactive Maps, "Leaflet Documentation and GIS Canvas Rendering Engine," 2024. [Online]. Available: https://leafletjs.com.

[8] Socket.IO Documentation, "Bidirectional Low-Latency Event-Driven Communication Protocols for Node.js and Browser Clients," 2024. [Online]. Available: https://socket.io.

[9] MongoDB Atlas, "MongoDB Architecture and Best Practices for Timeseries Telemetry Collections and Geospatial 2dsphere Indexing," MongoDB Technical Documentation, 2023.

[10] Anna University, "Regulations and Project Guidelines for Bachelor of Engineering (B.E.) Degree Programmes," Anna University Curriculum Guidelines, Chennai, Tamil Nadu, India, 2023.